

打印输出

2020年8月25日 18:45

第 6 章 工厂模式

6.1 简单工厂模式

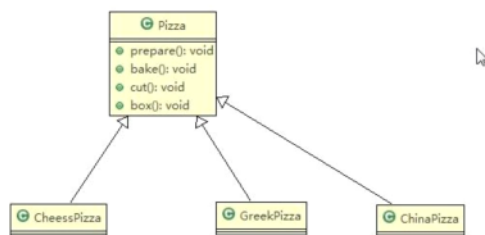
6.1.1 看一个具体的需求

看一个披萨的项目：要便于披萨种类的扩展，要便于维护

- 1) 披萨的种类很多(比如 GreekPizz、CheesePizz 等)
- 2) 披萨的制作有 prepare, bake, cut, box
- 3) 完成披萨店订购功能。

6.1.2 使用传统的方式来完成

- 1) 思路分析(类图)



编写 OrderPizza.java 去订购需要的各种 Pizza

- 2) 看老师代码的演示

```
public class OrderPizza {
```

```
// 构造器
// public OrderPizza() {
//     Pizza pizza = null;
//     String orderType; // 订购披萨的类型
//     do {
//         orderType = getType();
//         if (orderType.equals("greek")) {
//             pizza = new GreekPizza();
//             pizza.setName(" 希腊披萨 ");
//         } else if (orderType.equals("cheese")) {
//             pizza = new CheesePizza();
//             pizza.setName(" 奶酪披萨 ");
//         } else if (orderType.equals("pepper")) {
//             pizza = new PepperPizza();
//             pizza.setName("胡椒披萨");
//         } else {
//             break;
//         }
//         //输出 pizza 制作过程
//         pizza.prepare();
//         pizza.bake();
//         pizza.cut();
//         pizza.box();
//     } while (true);
// }
```

6.1.3 传统的方式的优缺点

- 1) 优点是比较好理解，简单易操作。
- 2) 缺点是违反了设计模式的 **ocp** 原则，即对扩展开放，对修改关闭。即当我们给类增加新功能的时候，尽量不修改代码，或者尽可能少修改代码。
- 3) 比如我们这时要新增加一个 **Pizza** 的种类(**Pepper** 披萨)，我们需要做如下修改。
如果我们增加一个 **Pizza** 类，只要是订购 **Pizza** 的代码都需要修改。

```
//增加一段代码 OrderPizza.java //写
if (ordertype.equals("greek")) {
    pizza = new GreekPizza();
} else if (ordertype.equals("pepper")) {
    pizza = new PepperPizza();
} else if (ordertype.equals("cheese")) {
    pizza = new CheesePizza();
} else {
    break;
}
```

4) 改进的思路分析

分析：修改代码可以接受，但是如果在其它的地方也有创建 **Pizza** 的代码，就意味着，也需要修改，而创建 **Pizza** 的代码，往往有多处。

思路：把创建 **Pizza** 对象封装到一个类中，这样我们有新的 **Pizza** 种类时，只需要修改该类就可，其它有创建到 **Pizza** 对象的代码就不需要修改了。→ 简单工厂模式

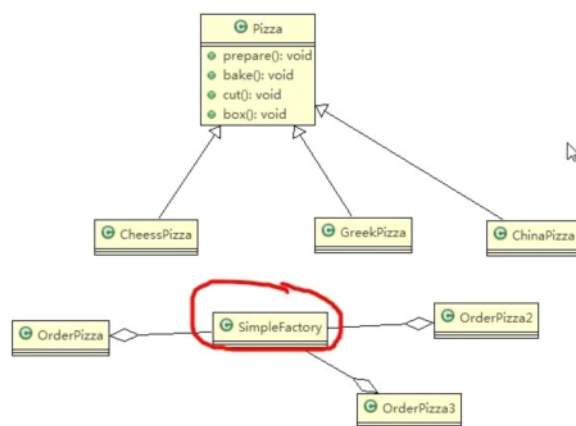
6.1.4 基本介绍

- 1) 简单工厂模式是属于创建型模式，是工厂模式的一种。简单工厂模式是由一个工厂对象决定创建出哪一种产品类的实例。简单工厂模式是工厂模式家族中最简单实用的模式

- 2) 简单工厂模式：定义了一个创建对象的类，由这个类来封装实例化对象的行为(代码)
- 3) 在软件开发中，当我们会用到大量的创建某种、某类或者某批对象时，就会使用到工厂模式。

6.1.5 使用简单工厂模式

- 1) 简单工厂模式的设计方案：定义一个可以实例化 **Pizza** 对象的类，封装创建对象的代码。



- 2) 看代码示例

```
package com.atguigu.factory.simplefactory.pizzastore.order;

import com.atguigu.factory.simplefactory.pizzastore.pizza.CheesePizza;
import com.atguigu.factory.simplefactory.pizzastore.pizza.GreekPizza;
import com.atguigu.factory.simplefactory.pizzastore.pizza.PepperPizza;
import com.atguigu.factory.simplefactory.pizzastore.pizza.Pizza;
```

```
//简单工厂类
public class SimpleFactory {

    //更加 orderType 返回对应的 Pizza 对象
    public Pizza createPizza(String orderType) {

        Pizza pizza = null;

        System.out.println("使用简单工厂模式");
        if (orderType.equals("greek")) {
            pizza = new GreekPizza();
            pizza.setName(" 希腊披萨 ");
        } else if (orderType.equals("cheese"))
        { pizza = new CheesePizza();

            pizza.setName(" 奶酪披萨 ");
        } else if (orderType.equals("pepper"))
        { pizza = new PepperPizza();

            pizza.setName("胡椒披萨");
        }

        return pizza;
    }

    //简单工厂模式 也叫 静态工厂模式
```

```
public static Pizza createPizza2(String orderType) {

    Pizza pizza = null;

    System.out.println("使用简单工厂模式 2");
    if (orderType.equals("greek")) {
        pizza = new GreekPizza();
        pizza.setName(" 希腊披萨 ");
    } else if (orderType.equals("cheese"))
    {
        pizza = new CheesePizza();
        pizza.setName(" 奶酪披萨 ");
    } else if (orderType.equals("pepper"))
    {
        pizza = new PepperPizza();
        pizza.setName("胡椒披萨");
    }

    return pizza;
}

}
```

```
//OrderPizza.java
```

```
package com.atguigu.factory.simplefactory.pizzastore.order;

import java.io.BufferedReader;
```

```
import java.io.IOException;
import java.io.InputStreamReader;

import com.atguigu.factory.simplefactory.pizzastore.pizza.Pizza;

public class OrderPizza {

    // 构造器
    // public OrderPizza() {
    //     Pizza pizza = null;
    //     String orderType; // 订购披萨的类型
    //     do {
    //         orderType = getType();
    //         if (orderType.equals("greek")) {
    //             pizza = new GreekPizza();
    //             pizza.setName(" 希腊披萨 ");
    //         } else if (orderType.equals("cheese")) {
    //             pizza = new CheesePizza();
    //             pizza.setName(" 奶酪披萨 ");
    //         } else if (orderType.equals("pepper")) {
    //             pizza = new PepperPizza();
    //             pizza.setName("胡椒披萨");
    //         } else {
    //             break;
    //         }
    //     }
    // }
```

```
//      //输出 pizza 制作过程
//      pizza.prepare();
//      pizza.bake();
//      pizza.cut();
//      pizza.box();
//
//      } while (true);
//  }

//定义一个简单工厂对象
SimpleFactory simpleFactory;
Pizza pizza = null;

//构造器
public OrderPizza(SimpleFactory simpleFactory)
    { setFactory(simpleFactory);
    }

public void setFactory(SimpleFactory simpleFactory) {
    String orderType = ""; //用户输入的

    this.simpleFactory = simpleFactory; //设置简单工厂对象

    do {
        orderType = getType();
        pizza = this.simpleFactory.createPizza(orderType);
    }
```

```
//输出 pizza
if(pizza != null) { //订购成功
    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
} else {
    System.out.println(" 订购披萨失败 ");
    break;
}
}while(true);
}

// 写一个方法，可以获取客户希望订购的披萨种类
private String getType()
{ try {
    BufferedReader strin = new BufferedReader(new InputStreamReader(System.in));
    System.out.println("input pizza 种类:");
    String str = strin.readLine();

    return str;
} catch (IOException e)
{ e.printStackTrace()
; return "";
}
}
```

```
}
```

6.2 工厂方法模式

6.2.1 看一个新的需求

披萨项目新的需求：客户在点披萨时，可以点不同口味的披萨，比如 北京的奶酪 pizza、北京的胡椒 pizza 或者是伦敦的奶酪 pizza、伦敦的胡椒 pizza。

6.2.2 思路1

使用简单工厂模式，创建不同的简单工厂类，比如 `BJPizzaSimpleFactory`、`LDAPizzaSimpleFactory` 等等。从当前这个案例来说，也是可以的，但是考虑到项目的规模，以及软件的可维护性、可扩展性并不是特别好

6.2.3 思路2

使用工厂方法模式

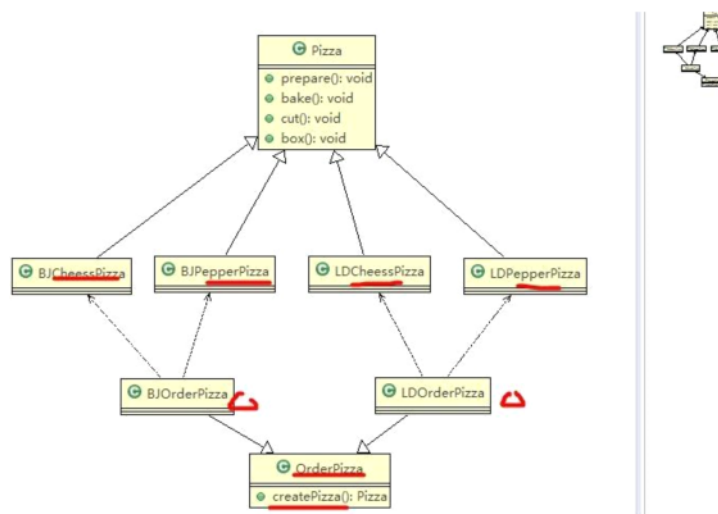
6.2.4 工厂方法模式介绍

- 1) 工厂方法模式设计方案：将披萨项目的实例化功能抽象成抽象方法，在不同的口味点餐子类中具体实现。
- 2) 工厂方法模式：定义了一个创建对象的抽象方法，由子类决定要实例化的类。工厂方法模式将对象的实例化推迟到子类。

6.2.5 工厂方法模式应用案例

1) 披萨项目新的需求：客户在点披萨时，可以点不同口味的披萨，比如 北京的奶酪 pizza、北京的胡椒 pizza 或者是伦敦的奶酪 pizza、伦敦的胡椒 pizza

2) 思路分析图解



3) 看老师代码实现

//OrderPizza.java 类

```

package com.atguigu.factory.factorymethod.pizzastore.order;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

import com.atguigu.factory.factorymethod.pizzastore.pizza.Pizza;
    
```

```
public abstract class OrderPizza {

    //定义一个抽象方法，createPizza，让各个工厂子类自己实现
    abstract Pizza createPizza(String orderType);

    // 构造器
    public OrderPizza()
    { Pizza pizza =
      null;

      String orderType; // 订购披萨的类型
      do {
          orderType = getType();
          pizza = createPizza(orderType); //抽象方法，由工厂子类完成
          //输出 pizza 制作过程
          pizza.prepare();
          pizza.bake();
          pizza.cut();
          pizza.box();

      } while (true);
    }
```

```
// 写一个方法，可以获取客户希望订购的披萨种类
private String getType()
{
    try {
        BufferedReader strin = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("input pizza 种类:");
        String str = strin.readLine();

        return str;
    } catch (IOException e)
    {
        e.printStackTrace();
        ; return "";
    }
}

}
```

```
package com.atguigu.factory.factorymethod.pizzastore.order;

import com.atguigu.factory.factorymethod.pizzastore.pizza.BJCheesePizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.BJPepperPizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.Pizza;

public class BJOrderPizza extends OrderPizza {
```

```
@Override
Pizza createPizza(String orderType) {

    Pizza pizza = null;
    if(orderType.equals("cheese")) {
        pizza = new BJCheesePizza();
    } else if (orderType.equals("pepper"))
        { pizza = new BJPepperPizza();
        }
    // TODO Auto-generated method stub
    return pizza;
}

}

package com.atguigu.factory.factorymethod.pizzastore.order;

import com.atguigu.factory.factorymethod.pizzastore.pizza.BJCheesePizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.BJPepperPizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.LDCheesePizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.LDPepperPizza;
import com.atguigu.factory.factorymethod.pizzastore.pizza.Pizza;

public class LDOrderPizza extends OrderPizza {
```

```
@Override
Pizza createPizza(String orderType) {

    Pizza pizza = null;
    if(orderType.equals("cheese")) {
        pizza = new LDCheesePizza();
    } else if (orderType.equals("pepper"))
    { pizza = new LDPepperPizza();
    }

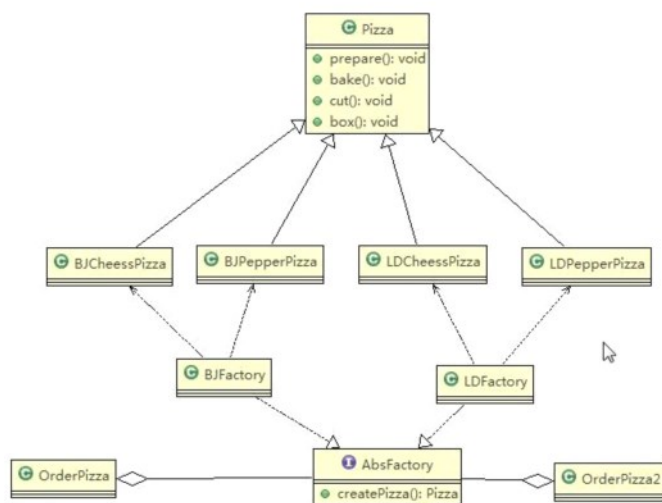
    // TODO Auto-generated method stub
    return pizza;
}

}
```

6.3 抽象工厂模式

6.3.1 基本介绍

- 1) 抽象工厂模式：定义了一个 **interface** 用于创建相关或有依赖关系的对象簇，而无需指明具体的类
- 2) 抽象工厂模式可以将简单工厂模式和工厂方法模式进行整合。
- 3) 从设计层面看，抽象工厂模式就是对简单工厂模式的改进(或者称为进一步的抽象)。
- 4) 将工厂抽象成两层，**AbsFactory**(抽象工厂) 和 具体实现的工厂子类。程序员可以根据创建对象类型使用对应的工厂子类。这样将单个的简单工厂类变成了工厂簇，更利于代码的维护和扩展。
- 5) 类图



6.3.2 抽象工厂模式应用实例

使用抽象工厂模式来完成披萨项目。

```
package com.atguigu.factory.absfactory.pizzastore.order;
```

```
import com.atguigu.factory.absfactory.pizzastore.pizza.Pizza;
```

```
//一个抽象工厂模式的抽象层(接口)
```

```
public interface AbsFactory {
```

```
    //让下面的工厂子类来 具体实现
```

```
    public Pizza createPizza(String orderType);
```

```
}
```

```
package com.atguigu.factory.absfactory.pizzastore.order;

import com.atguigu.factory.absfactory.pizzastore.pizza.BJCheesePizza;
import com.atguigu.factory.absfactory.pizzastore.pizza.BJPepperPizza;
import com.atguigu.factory.absfactory.pizzastore.pizza.Pizza;

//这是工厂子类
public class BJFactory implements AbsFactory {

    @Override
    public Pizza createPizza(String orderType) {
        System.out.println("~使用的是抽象工厂模式~");
        // TODO Auto-generated method stub
        Pizza pizza = null;
        if(orderType.equals("cheese")) {
            pizza = new BJCheesePizza();
        } else if
            (orderType.equals("pepper")){ piz
                za = new BJPepperPizza();
            }
        return pizza;
    }

}
```

```
package com.atguigu.factory.absfactory.pizzastore.order;
```

```
import com.atguigu.factory.absfactory.pizzastore.pizza.LDCheesePizza;
import com.atguigu.factory.absfactory.pizzastore.pizza.LDPepperPizza;
import com.atguigu.factory.absfactory.pizzastore.pizza.Pizza;
```

```
public class LDFactory implements AbsFactory {

    @Override
    public Pizza createPizza(String orderType) {
        System.out.println("~使用的是抽象工厂模式~");
        Pizza pizza = null;
        if (orderType.equals("cheese"))
            { pizza = new
              LDCheesePizza();
            } else if (orderType.equals("pepper"))
            { pizza = new LDPepperPizza();
            }
        return pizza;
    }

}
```

```
//OrderPizza.java
package com.atguigu.factory.absfactory.pizzastore.order;

import java.io.BufferedReader;
```

```
import java.io.IOException;
import java.io.InputStreamReader;

import com.atguigu.factory.absfactory.pizzastore.pizza.Pizza;

public class OrderPizza {

    AbsFactory factory;

    // 构造器
    public OrderPizza(AbsFactory factory)
    { setFactory(factory);
    }

    private void setFactory(AbsFactory factory)
    { Pizza pizza = null;

    String orderType = ""; // 用户输入
    this.factory = factory;
    do {
        orderType = getType();
        // factory 可能是北京的工厂子类，也可能是伦敦的工厂子类
        pizza = factory.createPizza(orderType);
        if (pizza != null) { // 订购 ok
            pizza.prepare();
            pizza.bake();
            pizza.cut();
        }
    }
}
```

```
        pizza.box();
    } else {
        System.out.println("订购失败");
        break;
    }
} while (true);
}

// 写一个方法，可以获取客户希望订购的披萨种类
private String getType()
{
    try {
        BufferedReader strin = new BufferedReader(new InputStreamReader(System.in));
        System.out.println("input pizza 种类:");
        String str = strin.readLine();

        return str;
    } catch (IOException e)
    {
        e.printStackTrace();
        ; return "";
    }
}
}
```

6.4 工厂模式在 JDK-Calendar 应用的源码分析

1) JDK 中的 Calendar 类中，就使用了简单工厂模式

2) 源码分析+Debug 源码+说明

➤ 源码部分

```
package com.atguigu.jdk;

import java.util.Calendar;

public class Factory {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        // getInstance 是 Calendar 静态方法
        Calendar cal = Calendar.getInstance();

        // 注意月份下标从 0 开始，所以取月份要+1

        System.out.println("年:" + cal.get(Calendar.YEAR));
        System.out.println("月:" + (cal.get(Calendar.MONTH) + 1));
        System.out.println("日:" + cal.get(Calendar.DAY_OF_MONTH));
        System.out.println("时:" + cal.get(Calendar.HOUR_OF_DAY));
        System.out.println("分:" + cal.get(Calendar.MINUTE));
        System.out.println("秒:" + cal.get(Calendar.SECOND));

    }

}

//Calendar.java
```

```
public static Calendar getInstance()
{
    return createCalendar(TimeZone.getDefault(), Locale.getDefault(Locale.Category.FORMAT));
}

private static Calendar createCalendar(TimeZone zone,
                                       Locale aLocale) //根据 TimeZone zone, locale 创建对应的实例
{
    CalendarProvider provider =
        LocaleProviderAdapter.getAdapter(CalendarProvider.class, aLocale)
            .getCalendarProvider();

    if (provider != null)
    { try {
        return provider.getInstance(zone, aLocale);
    } catch (IllegalArgumentException iae) {
        // fall back to the default instantiation
    }
    }

    Calendar cal = null;

    if (aLocale.hasExtensions()) {
        String caltype = aLocale.getUnicodeLocaleType("ca");
        if (caltype != null) {
```

```
switch (caltype)
{ case
"buddhist":
cal = new BuddhistCalendar(zone, aLocale);
break;
case "japanese":
cal = new JapaneseImperialCalendar(zone, aLocale);
break;
case "gregory":
cal = new GregorianCalendar(zone, aLocale);
break;
}
}
}
if (cal == null) {
// If no known calendar type is explicitly specified,
// perform the traditional way to create a Calendar:
// create a BuddhistCalendar for th_TH locale,
// a JapaneseImperialCalendar for ja_JP_JP locale, or
// a GregorianCalendar for any other locales.
// NOTE: The language, country and variant strings are interned.
if (aLocale.getLanguage() == "th" && aLocale.getCountry() == "TH")
{ cal = new BuddhistCalendar(zone, aLocale);
} else if (aLocale.getVariant() == "JP" && aLocale.getLanguage() == "ja"
&& aLocale.getCountry() == "JP") {
cal = new JapaneseImperialCalendar(zone, aLocale);
```



```
} else {
```

```
        cal = new GregorianCalendar(zone, aLocale);  
    }  
}  
return cal;  
}
```

6.5 工厂模式小结

1) 工厂模式的意义

将实例化对象的代码提取出来，放到一个类中统一管理和维护，达到和主项目的依赖关系的解耦。从而提高项目的扩展和维护性。

2) 三种工厂模式 (简单工厂模式、工厂方法模式、抽象工厂模式)

3) 设计模式的依赖抽象原则

- 创建对象实例时，不要直接 `new` 类, 而是把这个 `new` 类的动作放在一个工厂的方法中，并返回。有的书上说，变量不要直接持有具体类的引用。
- 不要让类继承具体类，而是继承抽象类或者是实现 `interface`(接口)
- 不要覆盖基类中已经实现的方法。